Object Detectors for Side-Scan Sonar Mine Detection

Stanford CS229 Project

Aditya Koushik
Dept. of Computer Science
Stanford University
akoushik@stanford.edu

Elvin Fu
Dept. of Electrical Engineering
Stanford University
elvinf@stanford.edu

Max Weinacht
Dept. of Physics
Stanford University
maxw2@stanford.edu

1 Introduction

Even after wars end, mines remain a lasting problem. Whether it is a local fishing boat or a container ship, sea mines often target the most innocent (just like in *Finding Nemo*). When neither side knows where they placed their mines - locally within a known mine field, or even regionally - machine learning techniques could provide vital insight into where to deploy limited de-mining resources.

2 Dataset and Features

Side-scan sonar (SSS) is a standard sensing method for scanning large seabed areas from autonomous underwater vehicles (AUVs). Compared to natural images, SSS imagery contains multiplicative speckle noise (from coherent acoustic scattering), range-dependent intensity falloff, characteristic highlight–shadow structures, and seabed textures with no clear natural-image analogs [1, 15]. COCO and ImageNet are large-scale natural-image benchmarks commonly used to pretrain detection and classification backbones, respectively. As a result, detectors pretrained on COCO or ImageNet must bridge a substantial domain gap during fine-tuning.

We focus primarily on the public SSS benchmark of Santos et al. [12], collected by the Portuguese Navy between 2010 and 2021. The benchmark includes two target classes: mine-like contacts (MILCO) and non-mine bottom objects (NOMBO), as well as many background-only images, making it a useful small, imbalanced evaluation target.

The dataset spans 1,170 side-scan images: 304 carry bounding-box annotations (437 MILCO and 231 NOMBO objects, ≈65%/35%) and 866 are unlabeled background (Figure S1). By per-image class composition the annotated images break down into 181 MILCO-only, 49 NOMBO-only, and 74 mixed. For every detection experiment except K-fold (Experiment 3), we use a stratified random 70/15/15 train/validation/test split based on four strata—background, MILCO-only, NOMBO-only, and mixed—independently. We prefer stratification over random split for uniform data representation across splits and to reduce variance across seeds. This also matches recent Santos detectors (e.g., ESL-YOLO and YOLOv11-SDC) which use a stratified random 7:2:1 split, and we adopt the same protocol for comparability [19, 20].

Figure 1 shows a representative Santos image with ground-truth bounding boxes. The object detection goal is to localize each target by predicting a bounding box and to classify it as MILCO (mine-like) or NOMBO (non-mine).

Our next dataset we examined was the BenthiCat (Harvard) dataset consisting of roughly one million SSS tiles collected along the coast

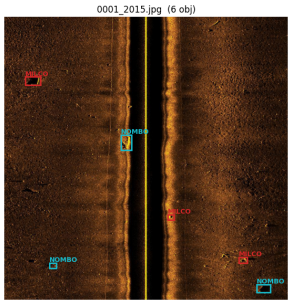

Figure 1: Example side-scan sonar image from the Santos benchmark with annotated targets.

of Catalonia (Spain). We use these undersea images for pretraining, not as a final test set, since BenthiCat does not include SSS scans with mine targets. The BenthiCat release is multimodal, pairing SSS tiles with additional sensing products (e.g., bathymetric maps) and co-registered optical imagery from targeted AUV surveys, and it includes 36,000 SSS tiles with manual segmentation masks for supervised fine-tuning.

Cylinder2 [18] was the final dataset we utilized, also containing side-scan sonar (SSS) imagery. These adopted for two-phase fine-tuning experiment. Cylinder2 contains 478 images across two classes: cylinder (338) and manta ray (140), with each image containing exactly one object. Each object is relatively small compared to the image size, making this dataset well suited to detecting small objects, such as mines or similar-looking underwater debris.

3 Related Work

Santos et al. [12] introduced the side-scan sonar mine-detection dataset used in this work and provided an initial YOLOv4 baseline of 75% mAP@0.5 (82% precision, 64% recall). The authors describe this as a preliminary, untuned result - as metrics were reported without a designated hold-out or validation set.

Earlier work on this dataset focuses on enhancing detection through specialized, sonar-specific architectural changes. Cui et al. [2] proposed SOCA-YOLO, combining SwinIR image restoration, SPDConv downsampling, and CBAM attention; they trained primarily on the Cylinder2 dataset and evaluated on Santos only as a cross-dataset generalization test. Zou et al. [20] proposed YOLOv11-SDC, which adds a Sparse Feature module, a Dilated Reparam Block, and Content-Guided Attention Fusion. Zhang et al. [19] proposed ESL-YOLO, integrating Sobel-based edge fusion, self-calibrated dual attention, and a location-quality estimator. This project's results are reported against prior detection methods in Table 5.

A separate work addresses the related but distinct task of patch-level classification. Kwon et al. [7] introduced Mine-JEPA and showed that in-domain self-supervised learning can outperform large vision foundation models for MILCO/NOMBO classification on this dataset. However, their setting assumes pre-localized patches and does not address whole-image object detection.

These gaps: no clean held-out protocol, an evaluation limited to pre-localized patches, and a reliance on custom architectural modules, motivate our approach. Rather than redesigning the detector for sonar, we keep the architecture fixed and ask how far *data and pretraining choices alone* close the natural-image-to-sonar domain gap, evaluated on a held-out test stratified split, which was not examined by previous methods. Because we change only the data and pretraining, the approach is not tied to any single architecture: it carries over to any detector, and to the stronger models that keep being released. We investigate two questions:

1. **Does sonar-specific data augmentation close the natural-image-to-sonar gap during fine-tuning?**
2. **How do different pretrained backbones and fine-tuning strategies transfer to SSS detection, and can sonar pretraining outperform large-scale natural-image pretraining?**

4 Methods

We use YOLOv8 [6], specifically the compact YOLOv8n variant ($\sim$3.2M parameters, with a convolutional CSPDarknet/C2f backbone). We choose a small convolutional detector over transformer-based detectors such as RT-DETRv2 [9] because the Santos et al. dataset [12] is quite small: a $\sim$3M-parameter model is easier to optimize from random or pretrained initialization and less prone to overfitting than larger models. With the detector architecture held fixed - YOLOv8n for the main initialization comparison, with a separate YOLO26n sub-study - the only variable across our initialization experiments is the source of training weights.

We organize the study into four experiments. Experiments 1 and 2 span the two axes *augmentation* and the source of *pretrained weights*; Experiment 3 performs stratified 5-fold cross-validation; and Experiment 4 is a classical, non-deep baseline. Experiment 1 sweeps data augmentation during fine-tuning, holding the initialization fixed at the default COCO-pretrained YOLOv8n. Beyond generic computer-vision augmentations (horizontal/vertical flips, translation, scaling, and mosaic; A1), we evaluate three physics-motivated sonar augmentations multiplicative speckle noise (A2), range-dependent intensity falloff (A3), and synthetic acoustic-shadow casting behind objects (A4), and combinations of augmentations. We carry the best-performing configuration into the remaining experiments and denote it $A^\star$.

Experiment 2 varies the source of pretrained weights with augmentation fixed at $A^\star$. On the YOLOv8n architecture we compare four weight sources: (i) random initialization (B1); (ii) a COCO-pretrained backbone+neck with a randomly initialized head (B2); (iii) the COCO-pretrained detector checkpoint carried over from Experiment 1 (B3; trained with augmentation $A^\star$); (iv) *self-supervised in-domain* pretraining, a YOLOv8n backbone trained with BYOL [4] on unlabeled BenthiCat side-scan-sonar tiles [13] (B4); and (v) *supervised in-domain transfer*, a detector first trained on the cylinder/manta side-scan-sonar dataset and then fine-tuned on Santos (B5). Finally, a YOLO26 sub-study reports a full YOLO26n COCO checkpoint (B6) and a higher-resolution YOLO26n-P2 variant initialized with a random detector (B7).

Experiment 3 re-evaluates the best system from Experiment 2 under a single stratified 5-fold cross-validation over the annotated images (using the same four strata as Section 2).

Finally, Experiment 4 provides a classification-only baseline that asks whether the backbone features linearly separate the two classes: each ground-truth box is cropped and embedded by a frozen, COCO-pretrained YOLOv8n backbone (layers 0–9, global-average-pooled to a 256-dimensional vector), and a logistic-regression classifier with balanced class weights predicts MILCO vs. NOMBO on a stratified split of the 668 object crops.

5 Experiments / Results / Discussion

5.1 Evaluation metrics (primary and secondary)

Detection metrics. We use the standard detection metrics computed by Ultralytics. A prediction counts as a true positive (TP) only if it is *both* of the correct class *and* localized with intersection-over-union (IoU) at threshold τ against an as-yet-unmatched ground-truth box (matches assigned in descending confidence order):

$$\mathrm{IoU}(B_p, B_{gt}) = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|} \geq \tau. \tag{1}$$

Otherwise it is a false positive (FP), and unmatched ground truths are false negatives (FN). Precision, recall, and F1 follow as

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \qquad \mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}, \qquad \mathrm{F1} = \frac{2\,\mathrm{P}\,\mathrm{R}}{\mathrm{P}+\mathrm{R}}. \tag{2}$$

Both depend on the confidence threshold; the single P and R we report are taken at the F1-maximizing threshold (Ultralytics convention). Average Precision at IoU τ is the area under the precision–recall curve,

$$\mathrm{AP}_\tau = \int_0^1 \mathrm{P}(\mathrm{R})\, d\mathrm{R}, \tag{3}$$

and mean Average Precision averages it over classes (mAP_τ). We report $\mathrm{mAP}_{0.5}$ (overall and per class for MILCO/NOMBO, given the class imbalance), the stricter $\mathrm{mAP}_{0.75}$, and the COCO-style

$$\mathrm{mAP}_{[0.5:0.95]} = \tfrac{1}{10} \sum_{\tau \in \{0.5,\ldots,0.95\}} \mathrm{mAP}_\tau, \tag{4}$$

which rewards tight localization and runs well below $\mathrm{mAP}_{0.5}$ for these small sonar targets.

Classification metrics (Experiment 4). For the logistic-regression experiment we report accuracy, class-conditional precision/recall, F1 (the harmonic mean of P and R), and AUC–ROC—the probability that a random positive is scored above a random negative (AUC $= 0.5$ is chance, 1.0 perfect).

5.2 Training protocol and hyperparameter selection

Chosen hyperparameters.

- Input resolution: 800×800 (above the Ultralytics 640 default, since mine-like objects are small).

Table 1: Summary of experiments. "Backbone Init" is the source of pretrained backbone (and neck) weights; "Head Init" is the source for the detection head; $A^\star$ is the augmentation configuration selected in Experiment 1. All detection experiments use the stratified random split described in Section 2, except Experiment 3, which uses a single stratified 5-fold cross-validation.

ID	Arch.	Backbone Init	Head Init	Augmentation
Experiment 1: Augmentation ablation (fixed COCO YOLOv8n init)				
A0	YOLOv8n	COCO	COCO	None
A1	YOLOv8n	COCO	COCO	Generic CV (flips, translate, scale, mosaic)
A2	YOLOv8n	COCO	COCO	Speckle noise
A3	YOLOv8n	COCO	COCO	Range-dependent falloff
A4	YOLOv8n	COCO	COCO	Acoustic shadow synthesis
A5	YOLOv8n	COCO	COCO	Physics stack (A2+A3+A4)
A6	YOLOv8n	COCO	COCO	Generic CV + physics stack
A7	YOLOv8n	COCO	COCO	Generic CV + speckle + range falloff
A8	YOLOv8n	COCO	COCO	All augmentations (A1+A2+A3+A4)
Experiment 2: Pretrained-weight source (augmentation fixed to $A^\star$)				
B1	YOLOv8n	Random	Random	$A^\star$
B2	YOLOv8n	COCO (backbone+neck)	Random	$A^\star$
B3	YOLOv8n	COCO (backbone+neck)	COCO†	$A^\star$
B4	YOLOv8n	Self-sup. BenthiCat SSS [13]	Random	$A^\star$
B5	YOLOv8n	Cylinder/manta SSS (supervised)	Cyl./manta†	$A^\star$
B6	YOLO26n	COCO (backbone+neck)	COCO†	$A^\star$
B7	YOLO26n-P2	COCO partial transfer	Random	$A^\star$
Experiment 3: Stratified k-fold cross-validation ($k = 5$)				
C1	YOLOv8n	Best of Exp. 2	Best of Exp. 2	$A^\star$ (5-fold stratified CV)
Experiment 4: Classical baseline (classification probe)				
D1	YOLOv8n (frozen)	COCO (layers 0–9)	—	256-d embedding $\rightarrow$ logistic regression

† The pretrained classification branch is re-initialized for the two Santos classes at fine-tuning time; box-regression weights carry over.
Row B3 (Experiment 2) is the COCO-pretrained detector checkpoint carried over from Experiment 1 (trained with augmentation $A^\star$) and reused here rather than retrained.

- Batch size: 16 (fits GPU memory at 800^2 and keeps batch-norm statistics stable).
- Optimizer: `auto`, which Ultralytics resolves to MuSGD for our long (>10k-iteration) schedule, with $\mathrm{lr}_0 = 0.01$, momentum 0.9, weight decay 5×10^{-4}.
- Learning rate & layer freezing: chosen from the grid in Sup. Fig. S4. Full fine-tuning (no frozen layers) at $\mathrm{lr}_0 = 0.01$ gives the best validation mAP@0.5.
- Epochs / patience: 1000 / 50. The 1000-epoch cap is intentionally high so that early stopping (patience 50 on validation mAP), not the cap, ends training.

How we chose them. We tuned only the base learning rate and backbone freezing (grid in Sup. Fig. S4), selecting on validation mAP@0.5 and evaluating once on test; this was done after the augmentation ablation, and the gain is reflected from A6 to B3 (Table S1 to Table 3). Since `optimizer=auto` overrides `lr0`, the grid used explicit `optimizer=MuSGD`; full fine-tuning beat freezing the first ten layers at every rate, so we freeze nothing. Everything else stays at Ultralytics defaults (final-LR fraction 0.01, momentum 0.937, 3 warm-up epochs, linear LR, `deterministic` on); we change only the input size (800) and `patience` (50). At only 668 labeled objects, tuning every knob risks overfitting the validation split rather than improving generalization.

5.3 Experiment 1: Augmentation sweeps

We isolate the effect of augmentation with the architecture (YOLOv8n) fixed. A0 is the unaugmented baseline; the best configuration, the full stack (A6), is carried forward as A^* for Section 5.4. For augmentation definitions and implementation details, see Appendix 7.3.

Table 2: Validation detection metrics (mean $\pm$ std over 3 seeds)

Augmentation	mAP@0.5	mAP@[0.5:0.95]	P	R
None (A0)	0.505 ± 0.026	0.300 ± 0.026	0.637 ± 0.198	0.515 ± 0.091
Generic CV (A1)	0.577 ± 0.101	0.331 ± 0.044	0.693 ± 0.011	0.583 ± 0.088
Speckle (A2)	0.572 ± 0.010	0.3634 ± 0.022	0.750 ± 0.082	0.504 ± 0.018
Range falloff (A3)	0.514 ± 0.028	0.313 ± 0.038	0.686 ± 0.063	0.485 ± 0.014
Shadow (A4)	0.536 ± 0.014	0.311 ± 0.018	0.751 ± 0.112	0.472 ± 0.016
Physics-only stack (A5)	0.568 ± 0.046	0.376 ± 0.034	$\mathbf{0.773 \pm 0.052}$	0.508 ± 0.007
Full stack (A6*)	$\mathbf{0.650 \pm 0.035}$	$\mathbf{0.377 \pm 0.016}$	0.758 ± 0.073	0.581 ± 0.029
Full w/o shadow (A7)	0.639 ± 0.023	0.363 ± 0.013	0.706 ± 0.022	$\mathbf{0.605 \pm 0.029}$

Physics-only augmentation (A5) improves over the no-augmentation baseline (A0) by $+0.063$ mAP@0.5 (0.568 vs. 0.505), indicating that sonar-motivated effects (speckle, range falloff, and shadow) add useful invariances. However, physics-only is slightly below generic CV augmentation alone (A1) by 0.009 mAP@0.5 (0.568 vs. 0.577), suggesting that geometry/composition changes (e.g., flips, scale/translate, mosaic) are the larger driver on this small dataset. The best results come from combining both families: the full stack (A6) reaches 0.650

mAP@0.5, a gain of $+0.073$ over generic CV (A1) and $+0.145$ over no augmentation (A0). This makes sense because the generic transforms encourage detector invariance to pose and placement, while the physics-based transforms match SSS-specific intensity and shadow statistics; together they reduce overfitting to the limited training distribution.

Mosaic probability. Mosaic tiles four images into one composite (adjusting bounding boxes accordingly), exposing the detector to varied object scales, positions, and backgrounds. Mosaic $= 1.0$ and $= 0.25$ are statistically tied on mAP@0.5; we select 1.0 for A^* as it gives the best mAP@[0.5:0.95] (0.477) with the lowest seed variance (± 0.006), indicating more consistent detection (Table S1).

5.4 Experiment 2: B-class pretrained-weight source experiments

We hold augmentation fixed at $A^\star$ and vary only the source of the initial weights. B1–B3 trace how much of a COCO-pretrained detector to transfer (nothing; backbone only; the full detector). B4 and B5 test in-domain transfer: self-supervised (BYOL on BenthiCat tiles) versus supervised (a detector first trained on Cylinder2). B6–B7 swap the architecture to YOLO26n.

Table 4: Held-out test detection metrics for selected methods (mean $\pm$ std over 3 seeds)

Metric	B3	B4	B5
AP@0.5 (MILCO)	0.746 ± 0.093	0.701 ± 0.082	$\mathbf{0.778 \pm 0.082}$
AP@0.5 (NOMBO)	0.720 ± 0.019	0.697 ± 0.070	$\mathbf{0.744 \pm 0.031}$
mAP@0.5	0.733 ± 0.037	0.699 ± 0.055	$\mathbf{0.761 \pm 0.043}$
Best mAP@0.5	0.772	0.779	**0.788**
mAP@[0.5:0.95]	0.523 ± 0.045	0.472 ± 0.059	$\mathbf{0.548 \pm 0.015}$
Best mAP@[0.5:0.95]	**0.573**	0.544	0.563
mAP@0.75	0.591 ± 0.004	0.521 ± 0.089	$\mathbf{0.631 \pm 0.017}$
Best mAP@0.75	0.594	0.617	**0.643**

Supervised in-domain transfer is the only clear winner. B5 (Cylinder→Santos) is best on both validation and the held-out test set across almost all metrics (test mAP@0.5 0.761 vs. 0.733 for COCO; highest validation precision 0.886 ± 0.018; Table 4). Its advantage likely comes from two-stage fine-tuning (Cylinder2 then Santos): the first stage warms up the detector head and box-regression features on small, high-contrast highlight–shadow targets, and the second adapts the SSS objectness/localization weights to Santos' classes and backgrounds.

The other YOLOv8 variants (B1–B4) sit within one seed standard deviation on mAP@0.5, so their ordering is not significant. Random init (B1, 0.699) is already competitive with full COCO transfer, likely because the small model and heavy augmentation limit how much a natural-image prior adds. Where COCO does help is localization: the COCO-backbone settings (B2, B3) reach the highest mAP@[0.5:0.95] (0.488, 0.496), placing tighter boxes despite mid-tier mAP@0.5.

In-domain self-supervision underperforms. B4 (BYOL on BenthiCat) does not beat COCO (test mAP@0.5 0.699 vs. 0.733) despite being sonar data: BenthiCat is seafloor/habitat texture with no mine-like objects and a different sonar, BYOL has no detection or classification objective, and only the backbone transfers (random head). B4 is also the highest-variance YOLOv8—its best seed (0.779) matches B5's (0.788) but its mean is lower.

The newer architecture does not help. YOLO26n (B6) underperforms compared to the YOLOv8n variants on validation (0.685 vs. 0.707–0.718). This could be becuase on a dataset this small, its added complexity is harder to optimize, and the simpler YOLOv8n with DFL loss localizes better. The P2 variant (B7) is worst by a wide margin (0.593 mAP@0.5, 0.353 mAP@[0.5:0.95], ± 0.10). P2's extra resolution targets small objects, but the added parameters need more data than $\sim$213 images can supply, so the capacity hurts more than the small-object focus helps.

5.5 Experiment 3: Stratified k-fold evaluation

To test whether our conclusions depend on one lucky test split, we take the best system from Experiment 2 (B5, Cylinder→Santos) and re-evaluate it under stratified 5-fold cross-validation. The mean test mAP@0.5 of 0.707 closely matches B5's validation score (0.718, Table 3), confirming the fixed split was not just lucky. The key finding is the variance: fold mAP@0.5 ranges from 0.548 to 0.860, with larger per-class swings (NOMBO AP 0.517 to 0.917). This is a property of the dataset, not the model: with only 304 annotated images split five ways, each fold has $\sim$60 test images, and objects are unevenly distributed across years (NOMBO is almost entirely in 2015—175 of 231 instances—with 0 in 2021 and 2 in 2017). Stratifying by class composition balances the count of each image type per fold but cannot balance a fold whose NOMBO images all come from one year. Single-split numbers thus carry wide uncertainty, and cross-validated means are more honest.

5.6 Comparison to prior work

The main result is the gap between our untuned starting point and our final configuration. Holding the YOLOv8n architecture fixed and changing only the training data weights, mAP@0.5 rises from 0.539 (default, no augmentation) to 0.761 (B5), an improvement of $+0.222$, while mAP@[0.5:0.95] rises from 0.350 to 0.548. Therefore, we believe sonar-specific augmentation and in-domain fine-tuning can close a lot of the natural-image-to-sonar gap, with no architectural changes and mostly default hyperparameters. Although B5 is below ESL-YOLO (0.847) and YOLOv11-SDC (0.825), there are caveats. These methods add custom sonar-specific modules (edge fusion, specialized attention) whereas we modify only the training data, so our approach transfers to any detector and to stronger future models. ESL-YOLO also reports a validation number, SOCA-YOLO uses Santos only as a cross-dataset test, and the original baseline defined no held-out split—all optimistic relative to a true held-out test. Even with our stricter split, our precision (0.882) exceeds ESL-YOLO's (0.877), and on the harder mAP@[0.5:0.95] our 0.548 is within both (0.584, 0.598).

Our untuned YOLOv8n (A0, 0.539) is below Cui et al.'s YOLOv9 baseline (0.743), likely not architecturally (YOLOv8n and YOLO26n differ little, Table 3) but because A0 disables augmentation and we use a stratified held-out split rather than the baseline's cross-dataset protocol. Prior detectors also report single-split, single-run numbers on a highly heterogeneous dataset (objects unevenly spread across years and seabed conditions, NOMBO almost entirely in one year); our 5-fold cross-validation (Table S2) shows the consequence—fold mAP@0.5 ranges from 0.55 to 0.86—so future work on this data should report stratified cross-validation.

Table 3: Validation metrics for pretrained-weight source comparisons (mean ± std over 3 seeds)

Setting	**mAP@0.5**	**mAP@[0.5:0.95]**	**P**	**R**
B1 (YOLOv8, random init)	0.699 ± 0.049	0.462 ± 0.059	0.819 ± 0.030	0.645 ± 0.081
B2 (YOLOv8, COCO backbone + random head)	0.689 ± 0.056	0.488 ± 0.060	0.828 ± 0.111	0.668 ± 0.022
B3 (YOLOv8, COCO detector checkpoint)	0.707 ± 0.043	$\mathbf{0.496 \pm 0.066}$	0.842 ± 0.035	$\mathbf{0.675 \pm 0.038}$
B4 (YOLOv8, BenthiCat init)	0.705 ± 0.064	0.473 ± 0.068	0.832 ± 0.066	0.629 ± 0.045
B5 (YOLOv8, Cylinder→Santos)	$\mathbf{0.718 \pm 0.034}$	0.491 ± 0.028	$\mathbf{0.886 \pm 0.018}$	0.652 ± 0.016
B6 (YOLO26, COCO weights)	0.685 ± 0.022	0.467 ± 0.027	0.822 ± 0.055	0.665 ± 0.012
B7 (YOLO26-P2, random detector)	0.593 ± 0.103	0.353 ± 0.101	0.706 ± 0.143	0.571 ± 0.075

Table 5: Comparison with published results on the Santos side-scan sonar mine dataset

Method	Base	P	R	$mAP_{0.5}$	$mAP_{0.5:0.95}$	Eval*
Published methods (architecture-modified)						
SOCA-YOLO [2]	YOLOv9	93.7	76.2	83.7	–	X-data
YOLOv9 baseline	YOLOv9	82.1	65.3	74.3	–	X-data
YOLOv11-SDC [20]	YOLOv11s	93.4	69.8	82.5	59.8	val/test
ESL-YOLO [19]	YOLOv11s	87.67	75.63	84.65	58.38	val
This work (architecture fixed; training data only)						
YOLOv8n default (A0)	no aug	63.7	51.5	53.9	35.0	val/test
Ours (B5, best)	Cyl.→Santos	88.2	65.2	76.1	54.8	val/test

* **X-data**: Santos used only as a cross-dataset generalization test (primary training set Cylinder2). **val**: headline on the validation split ("SIMD validation set"). **val/test**: split-vs-metric correspondence not stated unambiguously. **test**: held-out test split. Our rows are mean over 3 seeds; std omitted here for space (full values in Table 4).

5.7 Experiment 4: Logistic-regression baseline

Our logistic-regression baseline uses frozen COCO-pretrained YOLOv8 backbone features on cropped object regions: we crop the ground-truth boxes (with padding), embed them with the backbone, and train a class-balanced logistic regression to separate MILCO from NOMBO.

Performance (Table S4, Fig. S2) shows moderate separability: test accuracy 65.6% and AUROC 73.6%, above chance. MILCO is easier (78.9% precision, 68.2% recall, F1 = 0.732) than NOMBO (46.2% precision, 60.0% recall, F1 = 0.522), likely due to NOMBO's lower support and the MILCO-favoring class imbalance. The classifier does better on test than validation (73.6% vs 63.6%), indicating the frozen YOLOv8 features carry useful, non-trivial information for mine detection.

6 Conclusion / Future Work

This project evaluated side-scan sonar (SSS) mine detection by holding detector architecture fixed and isolating the effects of augmentation, pretraining, and evaluation protocol. Our strongest result was that training choices alone produced a substantial gain, where the baseline YOLOv8n system increased from 0.539 to 0.761 test mAP@0.5, and from 0.350 to 0.548 mAP@[0.5:0.95], without adding sonar-specific network modules. This makes our study useful not only as a Santos benchmark result, but as evidence that domain adaptation through data can perform well with more architecture-heavy approaches.

The best-performing detector was B5, the supervised Cylinder→Santos transfer model. It outperformed the COCO checkpoint and BenthiCat BYOL initialization on held-out mAP@0.5, mAP@0.75, and mAP@[0.5:0.95]. Its advantage is likely that Cylinder2 provides a supervised small-object sonar detection task before Santos fine-tuning, so the model enters the final stage with useful objectness and localization features rather than only generic natural-image features or texture-level sonar features. Our augmentation results support the same conclusion: generic geometric transforms made scale and placement robust, while speckle, range falloff and acoustic-shadow transforms helped match side-scan image performance. The full augmentation stack worked best because it addressed both limited data size as well as sonar-specific appearance.

A second conclusion is that evaluation protocol plays a key role in this specific dataset. The 5-fold experiment produced a mean mAP@0.5 of 0.707, but individual folds ranged from 0.548 to 0.860. This large spread suggests that single-split Santos results can overstate certainty, especially because class balance and collection year are uneven. Thus, this report's main contribution is not only its best model, but its clearer separation of architecture, data, and split uncertainty.

Future work should run more seeds, use cross-year validation, and test pretraining objectives closer to detection, such as segmentation, box prediction, or MILCO/NOMBO classification rather than BYOL alone. With more compute, we would also combine supervised sonar transfer with recent detector modules and use unlabeled Santos backgrounds for semi-supervised training. We can also explore vision transformer backbones such as DINOv2 [10] and DINOv3 [14] as well as vision transformer detectors such as RT-DETRv2 [9].

7 Appendices

7.1 Supplementary tables and figures

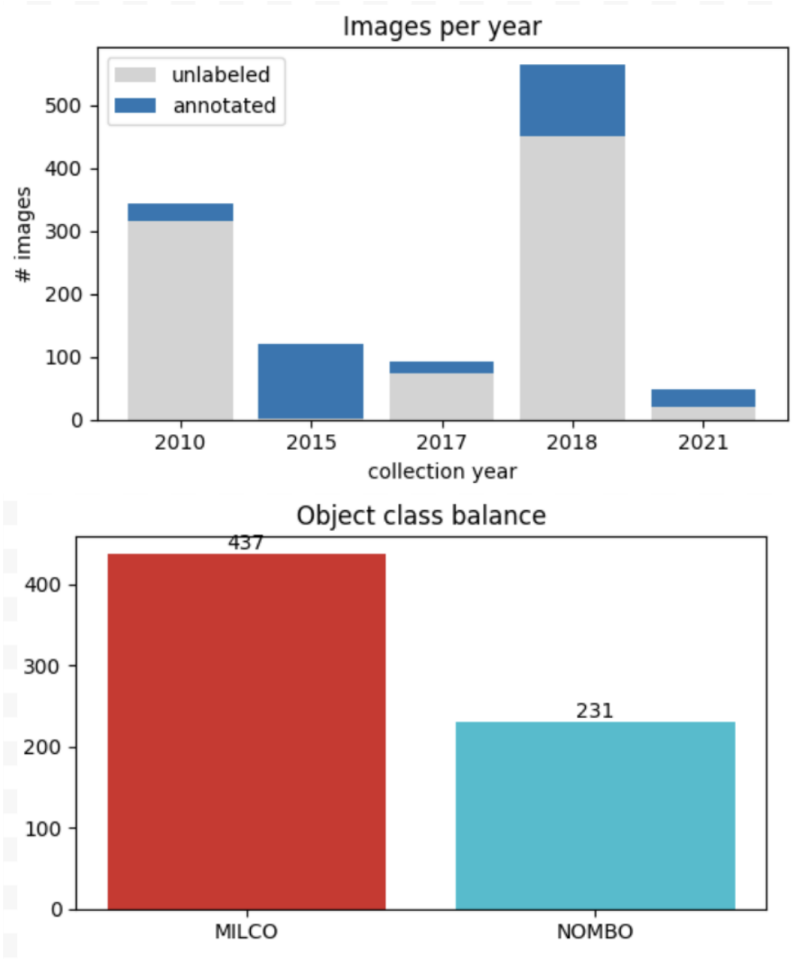

Figure S1: Composition of the Santos side-scan sonar benchmark

Table S1: Validation metrics for A6 while varying mosaic probability (mean ± std over 3 seeds)

Mosaic prob.	mAP@0.5	mAP@[0.5:0.95]	P	R
0.25	0.714 ± 0.075	0.472 ± 0.022	$\mathbf{0.860 \pm 0.001}$	$\mathbf{0.678 \pm 0.055}$
0.50	0.688 ± 0.002	0.408 ± 0.031	0.748 ± 0.003	0.661 ± 0.002
1.00	$\mathbf{0.719 \pm 0.046}$	$\mathbf{0.477 \pm 0.006}$	0.858 ± 0.010	0.656 ± 0.034

Table S2: Per-fold held-out test detection metrics

Fold	AP@0.5 (MILCO)	AP@0.5 (NOMBO)	mAP@0.5	mAP@[0.5:0.95]	mAP@0.75
0	0.803	0.917	0.860	0.609	0.686
1	0.738	0.622	0.680	0.479	0.558
2	0.651	0.659	0.655	0.402	0.408
3	0.578	0.517	0.548	0.371	0.379
4	0.826	0.755	0.791	0.526	0.598
Mean ± SD	0.719 ± 0.104	0.694 ± 0.151	0.707 ± 0.122	0.477 ± 0.096	0.526 ± 0.130

7.2 Models and objectives: YOLOv8, BYOL pretraining, and logistic regression

YOLOv8 detector. Our main detector is an Ultralytics YOLO model with adaptations to the SSS mine detection. Its CNN backbone consists of several Conb/C2f blocks to extract MILCO/NOMBO-specific patterns efficiently and deeply, also using Spatial Pyramid

Table S3: Per-class $AP_{0.5}$ on Santos

Method	$AP_{0.5}^{MILCO}$	$AP_{0.5}^{NOMBO}$	$mAP_{0.5}$
Prior baselines	–	–	(Table 5)
Ours (B5, best)	77.8 ± 8.2	74.4 ± 3.1	76.1 ± 4.3

	Class	Precision / %	Recall / %	F1	Support	accuracy / %	auroc / %
val	MILCO	72.7%	72.7%	0.727	22	0.625	63.6%
val	NOMBO	40.0%	40.0%	0.400	10		63.6%
val	macro avg	56.4%	56.4%	0.564	32		63.6%
val	weighted avg	62.5%	62.5%	0.625	32		63.6%
test	MILCO	78.9%	68.2%	0.732	22	0.656	73.6%
test	NOMBO	46.2%	60.0%	0.522	10		73.6%
test	macro avg	62.6%	64.1%	0.627	32		73.6%
test	weighted avg	68.7%	65.6%	0.666	32		73.6%

Table S4: Validation and test classification metrics.

Pooling (Fast) (SPPF) to allow generalization to different sizes andvariants of mines. YOLOv8's head consists of a PANet (Path Aggregation Network) that allows both the traditional top-down pathway (learned semantic information from the top layers of the backbone is upsampled and build into lower layers) as well as a bottom-up pathway enabled by a Feature Pyramind Network (FPN) that links the bottom layers of the CNN to higher ones preventing loss of sharp edges and the initial footprint possibly lost moving from layer to layer.

The YOLOv8 detector head works in an "anchor-free" manner (predicting boxes from features, not pre-determined locations; it needs to *find* the mine after all). Moreover, the the MILCO/NOMBO mine classification is done by seperate branches of the detector head, meaning the boxing and actual classification is done independently. This decoupled approach is more accurate as it prevents confusion between whether an possible object is even an object or is a mine. Finally, YOLOv8's loss follows that in Ultralytics implementation [6] that, first, uses Binary Cross-Entropy (BCE) on target assignment scores, minimizing the distance between our confidence in our estimate's and the truth: $L_{\text{cls}} = \frac{\sum \text{BCEWithLogits}(\hat{s},s)}{\sum s}$,

For learning boxes, YOLOv8 CloU (Complete Intersection over Union), rewarding boxes the more overlap exists, and DFL (Distribution Focal Loss), considering the boxes as a distribution over space:

$L_{\text{box}} = \frac{\sum(1-\text{CIoU}(\hat{b},b))w}{\sum s}$, $\quad L_{\text{dfl}} = \text{CE}(p_l, y_l)(y_r - y) + \text{CE}(p_r, y_r)(y - y_l)$, This makes the total loss for YOLOv8: $L = \lambda_{\text{box}} L_{\text{box}} + \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{dfl}} L_{\text{dfl}}$.

Another option was the YOLO26n detector, which uses COCO weights. YOLO26n-P2 adds even smaller detection head architecture beyond the P3/P4/P5 of YOLO26n, randomly initializing the new P2-specific layers that do not match with COCO. Ultimately, we opted for the YOLOv8n infrastructure because it performed better, likely due to simpler architecture and DFL, which aided bounding-box precision.

BYOL self-supervised pretraining. Our BOYL setup consisted of sending two augmented (cropping, flipping, speckle noise, blur, etc.) views of of each sonar tile through an online network and a target network. The online branch has encoder f_θ (image –> feature representation), projector g_θ (features –> representation space for self-supervised training), and predictor q_θ (predicts the target branch's representation); the target branch has f_ξ, g_ξ and no predictor. The target weights are an EMA (Exponential Moving Average) of the online weights:

$$\xi \leftarrow \tau\xi + (1-\tau)\theta.$$

The loss compares the online prediction to target representation using cosine similarity: $L_{\text{BYOL}} = 2 - 2\frac{q_\theta(g_\theta(f_\theta(v_1))) \cdot g_\xi(f_\xi(v_2))}{\|q_\theta(g_\theta(f_\theta(v_1)))\| \|g_\xi(f_\xi(v_2))\|} + (1 \leftrightarrow 2)$.

The augmentations we used were disabling color jitter (to avoid finding trivial patterns, as sonar images are usually black and white), translation, scaling, horizontal/vertical flips, and sonar-physics-specific transforms such as gamma speckle, range falloff, and acoustic shadow. To summarize, we use the pretrained YOLO backbone (through SPPF) and the neck and detect head are randomly initialized.

Logistic-regression baseline The logistic-regression experiment freezes model.model.model[:10] (YOLOv8 layers 0-9), skipping the neck and detect component. Each image is embedded by adaptive average pooling (AAP) to reduce the number of parameters

$$x = \text{flatten}(\text{AAP}(F_{0:9}(I))) \in \mathbb{R}^{256}.$$

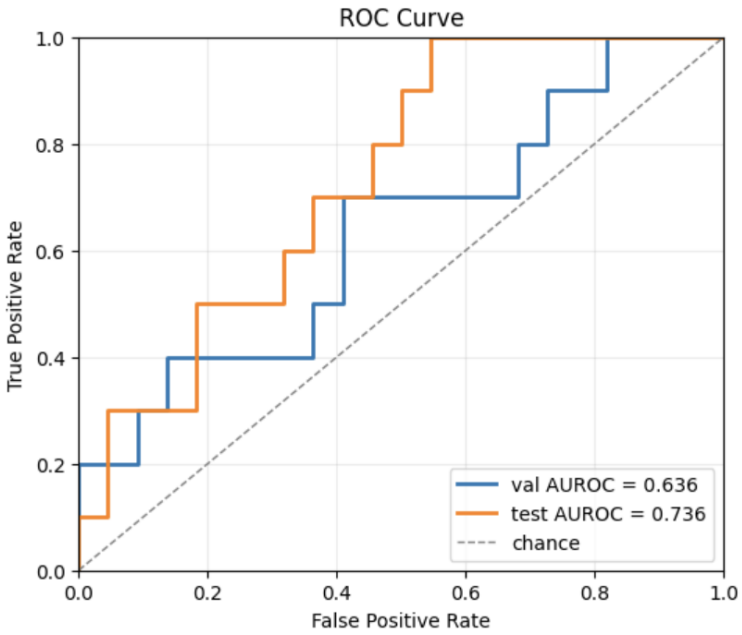

Figure S2: AUROC curves for the logistic-regression baseline.

A multinomial logistic model minimizes the following (softmax). This regression is also class-balanced, giving examples from rare classes a higher weight, so that the model does not just favor the majority class.

$$\min_W -\sum_i \alpha_{y_i} \log \frac{\exp(W_{y_i}^\top x_i)}{\sum_c \exp(W_c^\top x_i)} + \lambda \|W\|_2^2,$$

where $\alpha_c \propto 1/n_c$.

7.3 Augmentations

We group augmentations into (i) generic computer-vision transforms, applied through the Ultralytics pipeline, and (ii) three sonar-motivated transforms implemented as custom Albumentations operators. Unless noted, generic transforms use Ultralytics' default YOLOv8 probabilities.

Generic CV (A1). Standard geometric and photometric augmentations from the Ultralytics implementation: horizontal flip ($p = 0.5$), HSV jitter on hue/saturation/ value ($h = 0.015$, $s = 0.7$, $v = 0.4$), translation ($\pm 10\%$) and scaling ($\pm 50\%$), and mosaic, which tiles four images into one composite and adjusts the boxes accordingly (swept separately in Table S1). Vertical flip is enabled ($p = 0.5$) since side-scan imagery has no canonical up/down orientation. We keep the remaining defaults (e.g. no rotation or shear), which are already tuned for small-object COCO detection.

Speckle (A2). Multiplicative noise modeling coherent-acoustic speckle, $I' = I \cdot S$, with the field S sampled per pixel from a Rayleigh, Gamma, or log-normal distribution normalized to unit mean (so brightness is preserved on average); a width parameter σ controls noise strength. We use the Gamma field with $\sigma = 0.4$. Figure S3 shows the effect: fine granular texture is added across the seabed while the targets remain visible.

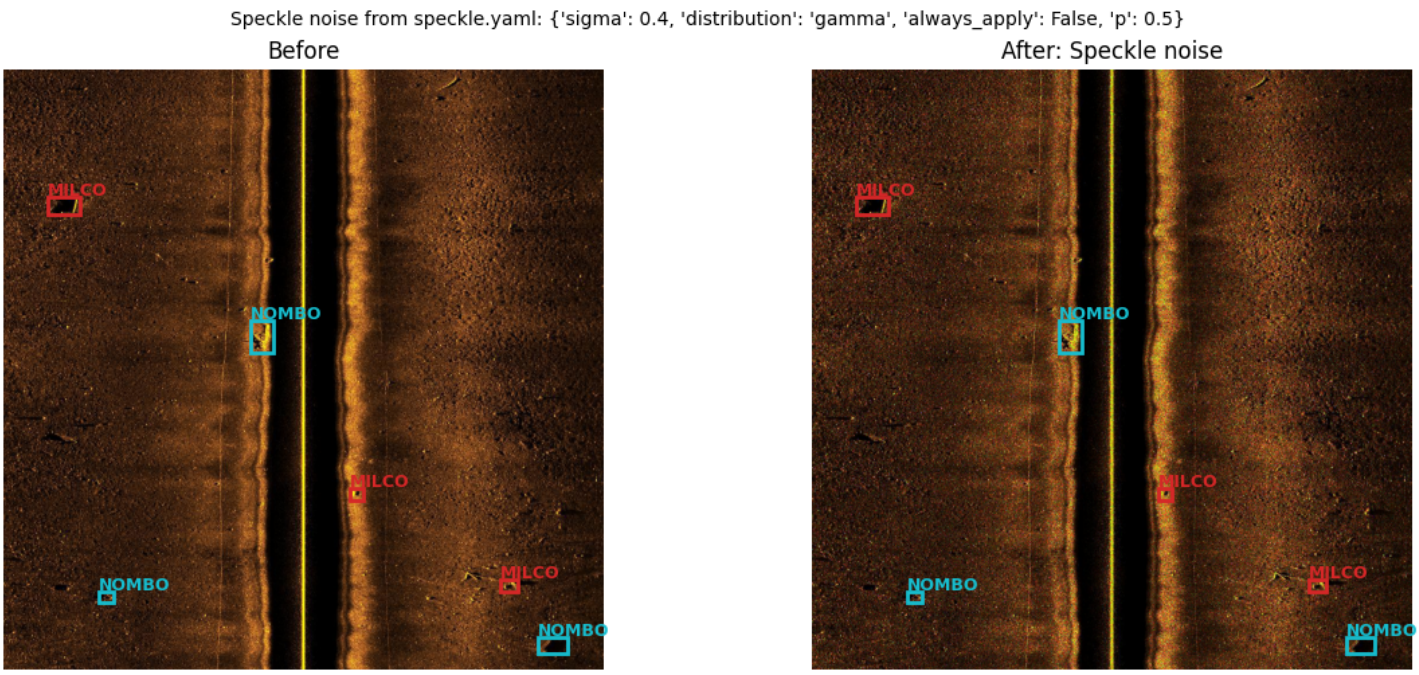

Figure S3: Speckle augmentation output (A2).

Range falloff (A3). A 1-D intensity roll-off along the range axis, $I'(r) = I(r) \cdot (1 + r)^{-\alpha}$, where r is normalized distance from the nadir line and the exponent α is sampled per image. This approximates distance-dependent attenuation and TVG over-/under-correction, dimming returns far from nadir.

Shadow (A4). Synthetic acoustic shadows behind annotated objects. For each box we darken a strip extending away from the nadir line (the acoustically occluded region), with a soft ramp from a dark object-adjacent edge back to full intensity at the far end. Shadow length is sampled per object as a fraction of image size; boxes are unchanged.

Stacks (A5–A8). Combinations of the above, as listed in Table 2.

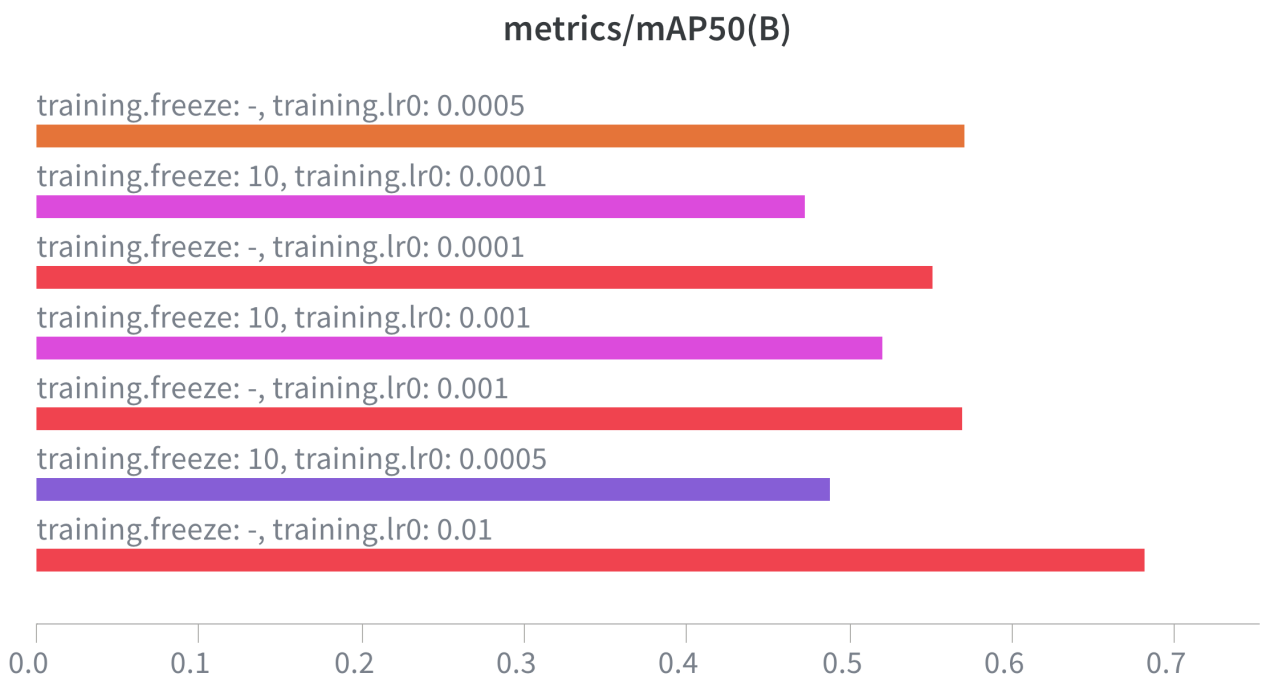

Figure S4: Learning-rate and layer-freeze sweep (validation mAP@0.5). Full fine-tuning (`freeze: -`) outperforms freezing the first ten layers (`freeze: 10`) at every learning rate, and peaks at $\mathrm{lr}_0 = 0.01$.

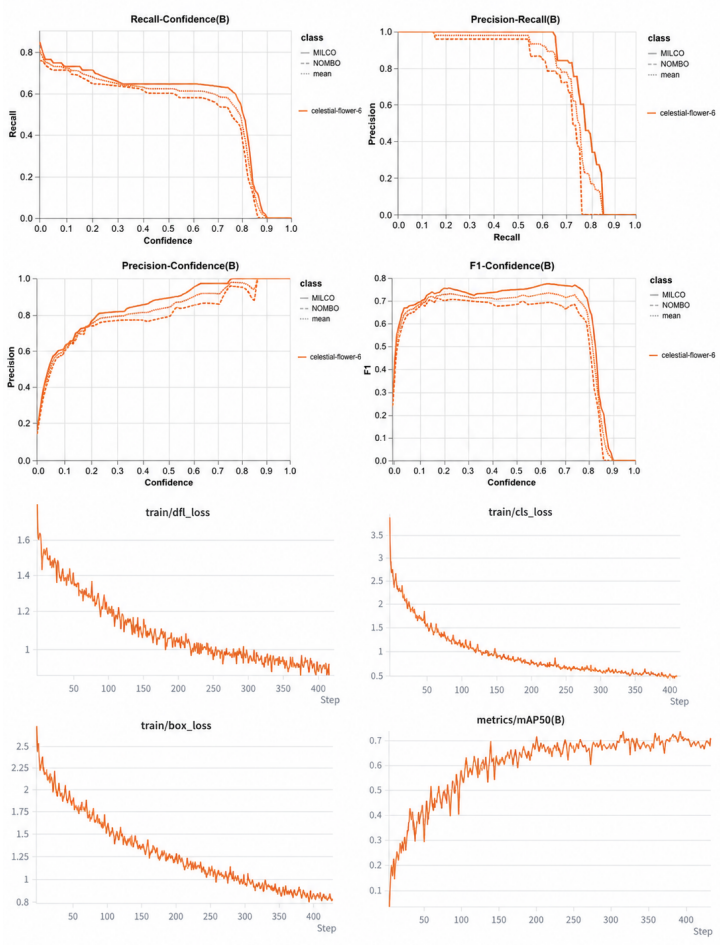

Figure S5: Training Curves from B5 Experiment (Best)

8 Contributions

Elvin Fu: ran the pretraining methods; contributed to the Results and Related Work sections.
Aditya Koushik: ran the fine-tuning methods; contributed to the Results section, the poster, and the writeup.
Max Weinacht: developed the logistic-regression baseline; wrote the Introduction, Related Work, and Sections 7.2, 5.5, and 5.7.

References

[1] Philippe Blondel. *The Handbook of Sidescan Sonar*. Springer Science & Business Media, 2010.

[2] Xiaodong Cui, Jiale Zhang, Lingling Zhang, Qunfei Zhang, and Jing Han. Small object detection in side-scan sonar images based on SOCA-YOLO and image restoration. *Frontiers in Marine Science*, 12:1542832, 2025. doi: 10.3389/fmars.2025.1542832.

[3] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *CVPR*, 2005.

[4] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *NeurIPS*, 2020.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[6] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8. `https://github.com/ultralytics/ultralytics`, 2023.

[7] Taeyoun Kwon, Youngwon Choi, Hyeonyu Kim, Myeongkyun Cho, Junhyeok Choi, and Moon Hwan Kim. Mine-JEPA: In-domain self-supervised learning for mine-like object classification in side-scan sonar. *arXiv preprint arXiv:2604.00383*, 2026.

[8] Yanghao Li, Hanzi Mao, Ross Girshick, and Kaiming He. Exploring plain vision transformer backbones for object detection. In *ECCV*, 2022.

[9] Wenyu Lv, Yian Zhao, Qinyao Chang, Kui Huang, Guanzhong Wang, and Yi Liu. RT-DETRv2: Improved baseline with bag-of-freebies for real-time detection transformer. *arXiv preprint arXiv:2407.17140*, 2024.

[10] Maxime Oquab et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.

[11] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

[12] Nuno Pessanha Santos, Ricardo Moura, Gonçalo Sampaio Torgal, Victor Lobo, and Miguel de Castro Neto. Side-scan sonar imaging data of underwater vehicles for mine detection. *Data in Brief*, 53:110132, 2024. doi: 10.1016/j.dib.2024.110132.

[13] Hayat Rajani, Valerio Franchi, Borja Martinez-Clavel Valles, Raimon Ramos, Rafael Garcia, and Nuno Gracias. BenthiCat: An opti-acoustic dataset for advancing benthic classification and habitat mapping. *arXiv preprint arXiv:2510.04876*, 2025.

[14] Oriane Siméoni et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025.

[15] Yannik Steiniger, Dieter Kraus, and Tobias Meisen. Survey on deep learning based computer vision for sonar imagery. *Engineering Applications of Artificial Intelligence*, 114:105157, 2022.

[16] Matias Valdenegro-Toro, Alan Preciado-Grijalva, and Bilal Wehbe. Pre-trained models for sonar images. *arXiv preprint arXiv:2108.01111*, 2021.

[17] Kaibing Xie, Jian Yang, and Kang Qiu. A dataset with multibeam forward-looking sonar for underwater object detection. *Scientific Data*, 9(1):739, 2022.

[18] yeesonmin@naver.com. cylider2 object detection dataset, version 6. Roboflow Universe, 2022. URL `https://universe.roboflow.com/yeesonmin-naver-com/cylider2/dataset/6`.

[19] Zhanshuo Zhang, Changgeng Shuai, Chengren Yuan, Buyun Li, Jianguo Ma, and Xiaodong Shang. ESL-YOLO: Edge-aware side-scan sonar object detection with adaptive quality assessment. *Journal of Marine Science and Engineering*, 13(8):1477, 2025. doi: 10.3390/jmse13081477.

[20] Chang Zou, Siquan Yu, Yankai Yu, Haitao Gu, and Xinlin Xu. Side-scan sonar small objects detection based on improved YOLOv11. *Journal of Marine Science and Engineering*, 13(1):162, 2025. doi: 10.3390/jmse13010162.